

Relatório de Auditoria de Segurança — Caderno de Estudos

Data: 09/09/2026

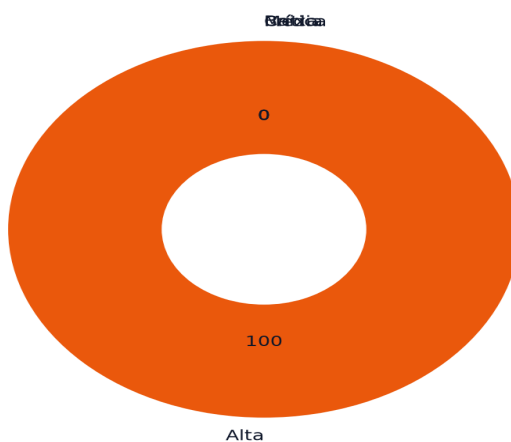
Escopo auditado: Vite + React + Firebase JS SDK (Firestore/Auth), sem backend, sem ORM, sem Docker/Helm/Terraform no repositório.

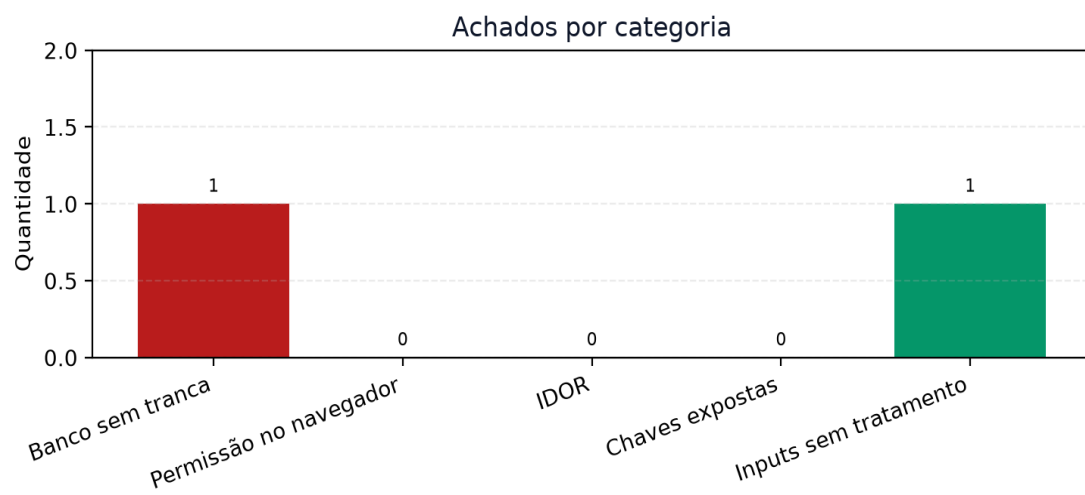
Nota metodológica: a auditoria mapou cada categoria à stack detectada; a categoria de isolamento foi tratada como Firestore Security Rules, as de permissão/IDOR foram avaliadas no contexto de ausência de backend e as de XSS foram verificadas nos fluxos do frontend.

Resumo executivo

Severidade	Quantidade
Crítica	0
Alta	2
Média	0
Baixa	0

Distribuição por severidade





Pontos fortes

- O código usa Firebase Auth e identifica usuário por ``user.uid`` em operações de escrita e leitura; isso sinaliza uma lógica de identidade clara e um desenho de dados por usuário, mesmo sem regras no repositório.
- O projeto não contém segredos embutidos em arquivos de código rastreados; as credenciais de Firebase presentes em `.env.local` são chaves Web públicas, não segredos de servidor.
- A aplicação não expõe bibliotecas de sanitização inadequadas e não há uso de `eval/new Function` no código auditado.

Pontos fracos

- Falta de Firestore Security Rules e de validação server-side no modelo de dados; tudo é escrito no cliente. Isso torna o isolamento de inquilinos dependente da configuração externa do Firebase e pode ser quebrado sem controle no código.
- A edição de notas aceita HTML bruto em ``ContentEditable``, sem sanitização. Isso configura risco de stored XSS e pode ser explorado por qualquer usuário que consiga inserir conteúdo malicioso em anotações.

Tabela de achados detalhados por categoria

Severidade	Arquivo:linha	Descrição
Alta	src/App.jsx:250-304	O app escreve diretamente em caminhos Firestore baseados em user.uid, mas r
Alta	src/App.jsx:944-1065	O editor ContentEditable persiste HTML bruto em Firestore e o re-renderiza sem

Recomendações priorizadas

1. P1 — Remover armazenamento de HTML bruto em notas e aplicar sanitização DOMPurify antes de persistir e renderizar.
2. P1 — Definir e versionar Firestore Security Rules com validação de owner/tenant e evitar acesso sem autenticação.
3. P2 — Adicionar camada de backend ou Cloud Functions para qualquer ação sensível, em vez de confiar em dados do cliente.
4. P3 — Separar configuração pública do Firebase (VITE_*) da configuração sensível e validar startup de variáveis necessárias.

ISSUES PARA O GITHUB

--- ISSUE 1 --- [Segurança] Falta de isolamento e regras de acesso no Firestore Labels sugeridas: security + alta Descrição do problema: A aplicação salva dados usando caminhos do tipo artifacts/<appId>/users/<user.uid>/notebooks/... diretamente no cliente. Em Firebase, esse isolamento só é garantido por Firestore Security Rules. Neste repositório não há regras de segurança, nem backend, o que torna a propriedade dos objetos dependente apenas do código front-end e do console do Firebase. Por que é explorável: Qualquer cliente autenticado, ou um ator que altere identificadores no front-end, pode escrever em caminhos que o código constrói com user.uid e activeNotebookId, sem uma checagem autoritativa no servidor. O código não valida no lado do servidor que o documento pertence ao usuário autenticado. Evidência: Arquivo: src/App.jsx:250-304; Trecho: await setDoc(doc(db, "artifacts", appId, "users", user.uid, "notebooks", activeNotebookId), updatedNotebook); Arquivo: src/config/firebase.js:5-16; Trecho: export const db = getFirestore(app); Impacto: Compromete o isolamento entre usuários e pode permitir leitura, escrita ou exclusão de dados de outros inquilinos/cuentas quando regras de Firestore não estiverem corretamente configuradas. Sugestão de correção: Definir Firestore Security Rules rigorosas com validação de request.auth.uid, resource.data.ownerUid e/ou tenant; rejeitar qualquer escrita fora do namespace do usuário autenticado; mover lógica sensível para Cloud Functions se necessário. Critérios de aceite: Existe firestore.rules versionado; toda escrita verifica request.auth.uid e o dono do documento; testes de segurança validam que um usuário não consegue escrever em outro user.uid. --- FIM ISSUE 1 ---

--- ISSUE 2 --- [Segurança] Stored XSS em notas via ContentEditable sem sanitização Labels sugeridas: security + alta Descrição do problema: O campo de edição de notas usa react-contenteditable com html={note.content || ""} e o conteúdo bruto é persistido em autoSaveNoteContent(note.id, e.target.value). Não existe sanitização antes do armazenamento nem antes do render; o HTML está sendo aceito e reemitido como DOM. Por que é explorável: Um usuário malicioso pode salvar nota contendo, por exemplo, ou outra marcação HTML ativa. Quando a nota é renderizada, a página executará o script/HTML no cliente do navegador do usuário autenticado. Evidência: Arquivo: src/App.jsx:944-1065; Trecho: ContentEditable html={note.content || ""} onChange={(e) => { autoSaveNoteContent(note.id, e.target.value); }} Arquivo: src/App.jsx:908-929; Trecho: ContentEditable innerRef={contentEditableRef} html={noteForm.content} onChange={(e) => setNoteForm({ ...noteForm, content: e.target.value })} Impacto: Stored XSS pode roubar sessão, manipular UI, persistir ações maliciosas e impactar outros usuários que abram as anotações. Sugestão de correção: Sanitizar todo HTML antes de salvar com DOMPurify ou equivalente; persistir texto simples em vez de HTML bruto quando possível; usar editor com sanitização obrigatória. Critérios de aceite: O HTML das notas é sanitizado antes do armazenamento; a renderização usa HTML limpo e validado; testes cobrem payloads como <script>, onerror e javascript:. --- FIM ISSUE 2 ---